
Loop Engineering

Construye tu Agente Autónomo —
APIs reales, no carpetas locales

CLAUDE PARA PRODUCTIVIDAD · NIVEL 2 · SESIÓN 4 DE 5
LEÓN RUIZ

Dónde quedamos — El viaje hasta hoy

En tres sesiones pasaron de "darle órdenes" a "darle un objetivo".

SESIÓN	LO QUE CONSTRUYERON	EN LENGUAJE DE LOOPS
S1	Fundamentos Mapearon su proceso y hablaron con el agente	Nace el Loop 1
S2	Harness & Skills Le dieron herramientas, skills y acceso real	Loop 1 robusto
S3	Orquestación Un objetivo → múltiples outputs y destinos, con decisiones propias	Loop 1 completo

Diagnóstico rápido

1 ¿Su agente de S3 corrió de principio a fin?
(Sí / No / Parcial)

2 ¿Alguna vez produjo algo que no esperaban?
(Tono incorrecto, longitud excedida, alucinación, error)

Casi todos levantan la mano en la segunda pregunta.
Eso es exactamente lo que hoy resolvemos.

Qué es Loop Engineering

De "prompt" a "sistema"

Loop Engineering es diseñar los **ciclos** que hacen a un agente confiable: que ejecute, **se verifique**, **recuerde** y **se dispare solo** — con infraestructura que persiste, no en una carpeta que vive en tu laptop.

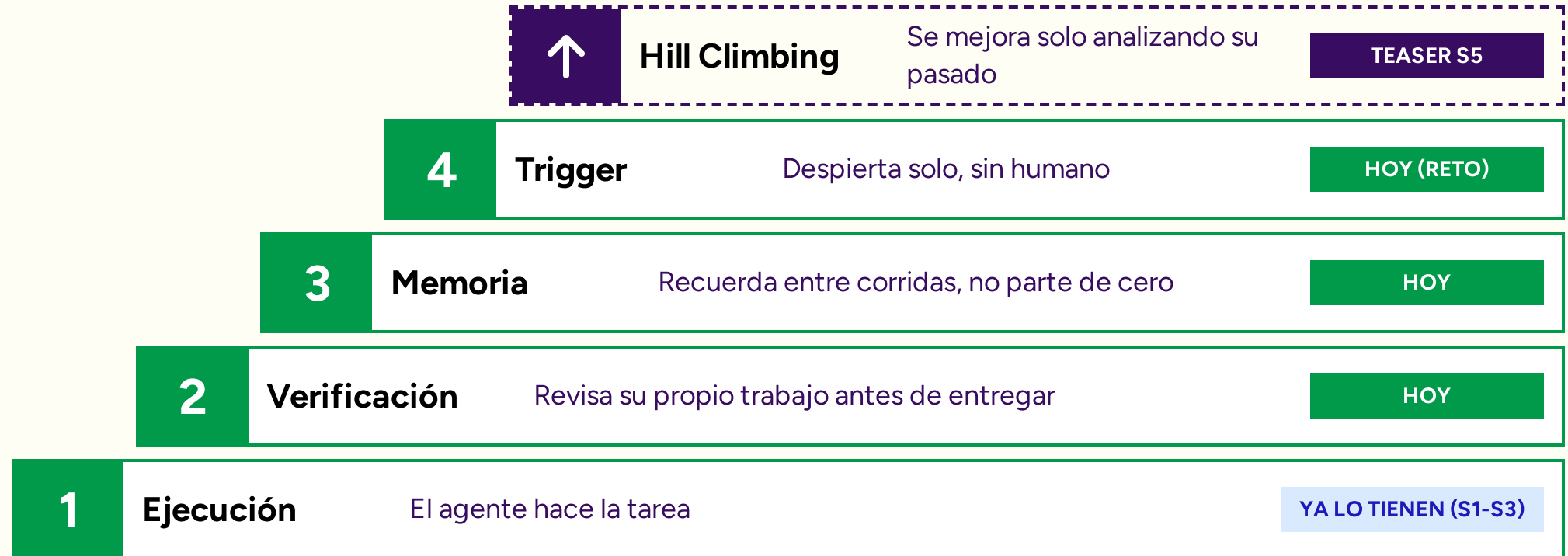
🏗️ S2 les dio el **harness**. S3 les dio la **orquestación**. S4 les da los **loops**.

👤 Un agente sin loops es un **asistente** que espera órdenes.

👤 Un agente con loops es un **Chief of Operations** real.

La escalera de los 4 loops

Cuatro niveles que se apilan para construir un sistema confiable.



Hoy subimos del Loop 1 al sistema completo — **y lo construyen ustedes en el hackathon.**

Loop 2 — Verificación

"Terminó" no es lo mismo que "lo hizo bien"

El Agent Loop termina cuando el agente decide que terminó — no cuando el resultado es correcto.



Exceder longitud

Le pidieron un resumen de un párrafo y entregó tres.



Tono incorrecto

Suena casual cuando requería formalidad, o robótico cuando necesitaba empatía.



Promesa no autorizada

Ofrece un reembolso o descuento que nadie aprobó.



Inconsistencia

El formato del output cambia sin razón aparente entre ejecuciones.

Pedirle "ten más cuidado" no funciona.

La solución: un segundo proceso que revisa antes de entregar.

Loop 2 — El patrón



1. FEEDBACK ESPECÍFICO

El worker no adivina qué corregir. El grader le dice exactamente por qué falló.

💬 "Tu respuesta tiene 142 palabras, el máximo es 60."

2. GRADER HÍBRIDO



Código (Gratis): Para lo objetivo y medible (longitud, frases prohibidas).



LLM Barato (Haiku): Solo para lo subjetivo que requiere juicio (tono, empatía).

Código para lo medible, LLM barato para el juicio.
Eso hace la verificación sostenible.

Primer: ¿Qué es una API?

Un "enchufe" para que tus programas hablen con servicios externos.



La Analogía

Imagina un restaurante:



Tú (Cliente): Quieres comida.



El Menú (API): Te dice qué puedes pedir y cómo pedirlo.

La Cocina (Servidor): Prepara la comida y te la entrega.

No entras a la cocina, solo usas el menú para obtener el resultado.



En nuestro Reto

Tu agente usa APIs para salir de Cowork:



OpenRouter: API para pensar (conecta con Claude).



Supabase: API para recordar (escribe en la base de datos).



Webhook: API para despertar (recibe el trigger).

Hoy su agente deja de estar aislado. Aprende a **enchufarse a servicios externos** para operar en el mundo real.

Las 4 herramientas del reto

El stack del hackathon, en una línea cada uno.



OpenRouter

Una puerta única a muchos modelos de IA (Claude, etc.)

El cerebro (Loop 2)



Supabase

Una base de datos en la nube con API automática

La memoria (Loop 3)



GitHub

Donde vive y se versiona el código

El entregable



n8n (opcional)

Orquestador visual de flujos en la nube

El disparador (Loop 4)

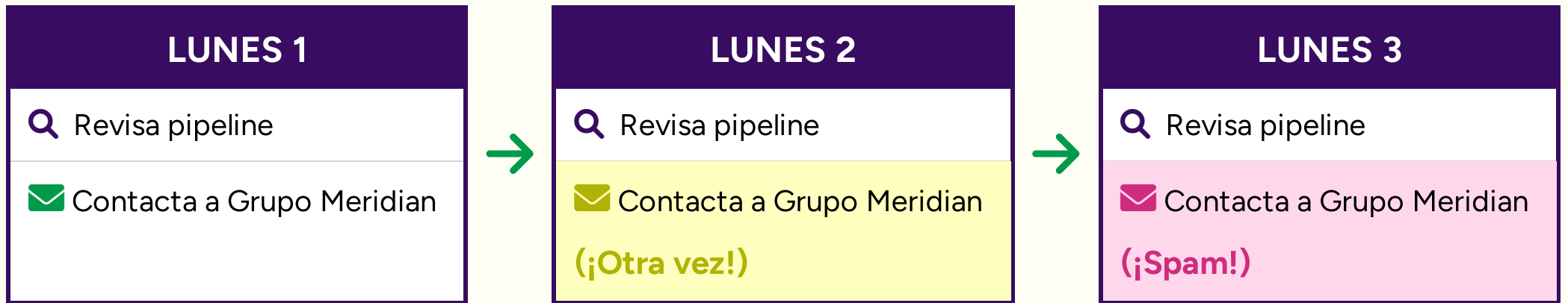
Su agente —vía Cowork o Manus— va a orquestar estas piezas.

Ustedes no programan desde cero: completan plantillas.

Loop 3 — Memoria

Su agente no recuerda nada. Cero.

Cada vez que el trigger dispara, el agente arranca desde cero. No sabe a quién ya contactó la semana pasada.



Sin memoria, manda el mismo email a las mismas cuentas semana tras semana.
El resultado: genera ruido e irrita a los clientes.

Loop 3 — De archivo local a base de datos

El `estado.json` evoluciona.

 ANTES (S3)

Archivo Local

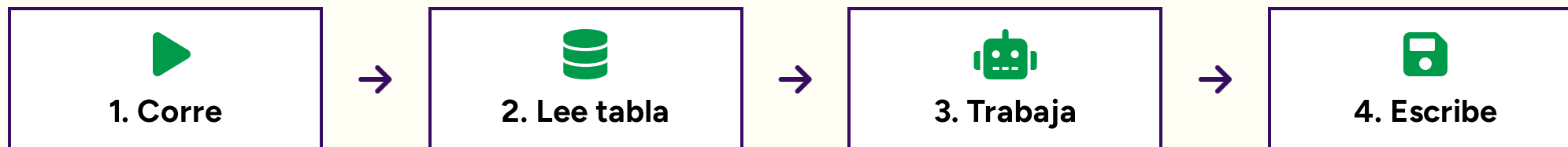
Un archivo que el agente lee al inicio y escribe al final. Vive aislado en tu computadora.

 AHORA (S4)

Tabla en Supabase

Misma idea, pero en la nube: auditable, compatible, y lista para integrarse con otras herramientas sin depender de tu laptop.

EL CICLO DE MEMORIA



Esto es "salir de lo local" que pidieron: su memoria deja de ser un archivo en tu compu y se vuelve **infraestructura real**.

Loop 4 — Trigger · Cron vs Webhook

¿Qué despierta al agente?

	 Cron	 Webhook
Qué es	Horario fijo (ej. cada lunes a las 8am)	Reacciona a un evento en tiempo real
Ejemplo	Reporte semanal, consolidado diario	Llega un formulario, nuevo registro urgente
Herramienta	Cowork / schedule	n8n / endpoint en la nube
Límite	Depende de tener Desktop abierto	Corre solo, en la nube

Si el momento exacto importa → **webhook**. Si puede esperar al lunes → **cron**.
Hoy, en el carril avanzado, lo construyen.

Demo de referencia

El Agente Autónomo en acción

Loop 4
Trigger



Webhook despierta al agente automáticamente.

Loop 1
Ejecución



Claude redacta mensajes personalizados para cada cuenta.

Loop 3
Memoria



Escribe los resultados y estatus de vuelta en **Supabase**.

Loop 3
Memoria



Lee **Supabase** para identificar cuentas inactivas (+30 días).

Loop 2
Verificación



Código + Haiku revisan longitud, tono y reglas de negocio.

Costo total de la corrida completa:

~\$0.012

El reto — El Agente Autónomo

1h 40min para construir un sistema real.

LA MISIÓN

Construir un agente que gestione la reactivación de cuentas inactivas usando infraestructura en la nube, sin intervención humana.

El Contexto

Mismo caso de uso (Grupo Meridian), pero esta vez con memoria real, verificación de calidad y autonomía total.

1

Detectar: Leer Supabase y filtrar cuentas con > 30 días inactivas.

2

Generar: Redactar un mensaje personalizado vía OpenRouter.

3

Verificar: Validar longitud y tono antes de darlo por bueno.

4

Registrar: Actualizar el estatus en Supabase (Memoria).

5

Disparar: Configurar un webhook para que despierte solo.

OPCIONAL

Elige tu carril

Tres niveles de dificultad. Nadie se queda atrás.

1. FUNDAMENTOS

L1: Ejecución

L3: Memoria

Conecta Supabase y OpenRouter. El agente lee las cuentas, genera el mensaje y actualiza la base de datos.

Meta: Lograr persistencia real.

2. VERIFICACIÓN

L1: Ejecución

L2: Verificación

L3: Memoria

Todo lo del Carril 1, más el grader híbrido. El agente debe rechazar sus propios errores y reintentar.

Meta: Lograr calidad autónoma.

3. AUTÓNOMO

L1

L2

L3

L4: Trigger

El sistema completo. Configuras un webhook (ej. en n8n) para que el agente despierte solo ante un evento.

Meta: Cero intervención humana.

Todos parten de una plantilla con instrucciones claras.
Si terminan rápido, suban al siguiente carril.

El stack y cómo arrancar

Preparando el entorno para el hackathon.

1. VARIABLES NECESARIAS

CEREBRO (OPENROUTER)
OPENROUTER_API_KEY

MEMORIA (SUPABASE)
SUPABASE_URL

AUTENTICACIÓN (SUPABASE)
SUPABASE_KEY

2. PASOS DE EJECUCIÓN

1

Clonar repo: Descargar las plantillas base.

2

Crear tablas: Ejecutar el esquema SQL en Supabase.

3

Cargar datos: Insertar las cuentas semilla (Meridian, etc).

4

Correr carril: Completar el código y ejecutar el agente.



Red de seguridad: Si alguien tiene problemas creando su cuenta de Supabase, tenemos un clon local listo en el entorno para que nadie se quede atrás.

Reglas del hackathon

El marco de trabajo para los próximos 100 minutos.



1. EQUIPOS

Trabajen en grupos de 2 a 3 personas. Apóyense para resolver bloqueos técnicos.



2. CARRIL Y PLANTILLAS

Elijan su carril (1, 2 o 3) y arranquen **estrictamente** desde la plantilla correspondiente.



3. ENTREGABLE

El resultado final debe vivir en un repositorio de GitHub con un README claro.



4. SEGURIDAD

Usen llaves de API (OpenRouter) con un límite de gasto bajo configurado (\$1-\$5).



5. DEMO FINAL

Tendrán 2-3 minutos por equipo al terminar el tiempo para presentar su agente.



6. FOCO DEL DEMO

Muestren qué loops lograron conectar y qué cambió físicamente en Supabase.

El objetivo no es que el código sea perfecto a la primera.

El objetivo es entender cómo se conectan las piezas del sistema.

Rúbrica

Cómo evaluaremos el éxito del agente.

CRITERIO	DESCRIPCIÓN	PESO
 Loop 3: Memoria	Lee correctamente de Supabase y actualiza el estatus al finalizar.	25%
 Loop 2: Verificación	Implementa el grader para rechazar mensajes que no cumplen las reglas.	25%
 Loop 1: Ejecución	Genera mensajes personalizados y coherentes vía OpenRouter.	20%
 Loop 4: Trigger	Configura un webhook funcional para despertar al agente.	15%
 Entregable	Repositorio en GitHub con código limpio y README explicativo.	15%



Nota para el Carril 1: Los pesos de Verificación y Trigger se redistribuyen hacia Ejecución y Memoria. **Elegir un carril accesible no penaliza la calificación.**

Checkpoints

Asegurando que nadie se quede atrás.

⌚ MINUTO 30



Checkpoint 1

¿Quién ya logró conectar y leer datos desde Supabase?

⌚ MINUTO 60



Checkpoint 2

¿Quién ya está escribiendo resultados o verificando con el grader?

☕ FLEXIBLE



Break 10 min

Pausa para estirar las piernas, tomar agua y despejar la mente.

Haremos estas pausas grupales para compartir bloqueos y soluciones.

Si un equipo se atora, lo resolvemos juntos.

Demos de equipos

Es hora de ver a sus agentes en acción.

 Seleccionaremos **3 a 4 equipos** para presentar sus resultados.

 Tendrán **2 a 3 minutos** por equipo para mostrar su sistema.

 Priorizaremos mostrar una **variedad de carriles** (Fundamentos, Verificación, Autónomo).

PREGUNTAS GUÍA

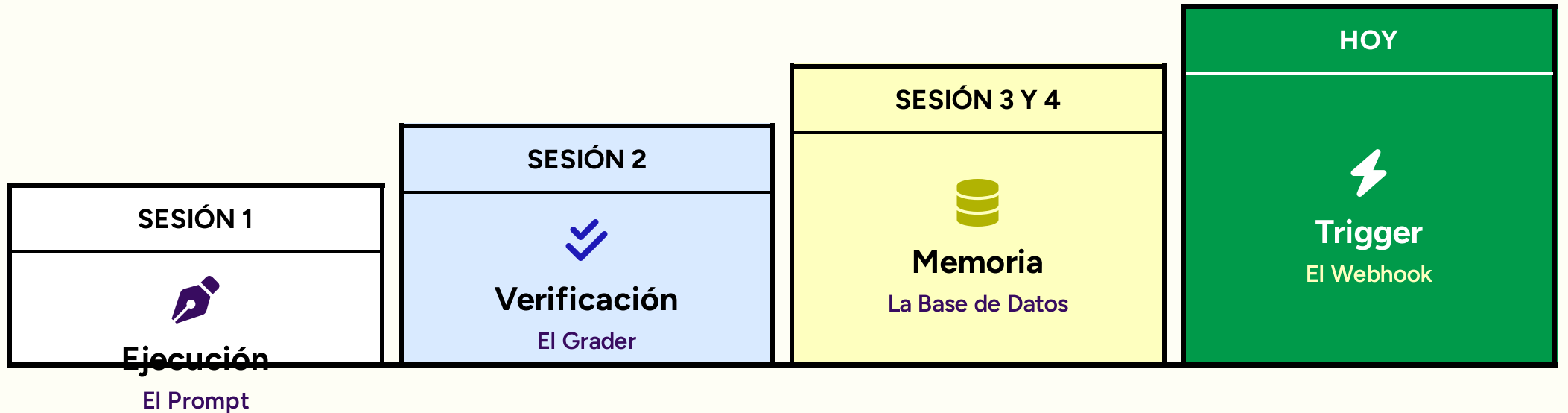
1 ¿Qué **carril** eligieron y por qué?

2 ¿Qué **loop** fue el más difícil de conectar o entender?

3 ¿Qué cambió físicamente en su tabla de **Supabase**?

La escalera completa

De un prompt a un Agente Autónomo.



PRÓXIMA SESIÓN: HILL CLIMBING



¿Qué pasa cuando el sistema analiza sus propios resultados y **se mejora solo** sin que tú diseñes las reglas?

Pre-work S5 + Cierre

El siguiente paso en su evolución.



1. ENTREGABLE

Asegúrense de hacer push de su código final a GitHub. Será la base sobre la que construiremos en la Sesión 5.



2. REFLEXIÓN

Piensen: ¿Qué proceso real de su empresa migrarían a este stack (Memoria + Verificación + Autonomía)?

Hoy cruzaron el umbral. Dejaron de tener un asistente local y pasaron a construir un sistema que **ejecuta, revisa, recuerda y despierta solo.**

BIENVENIDOS AL LOOP ENGINEERING.